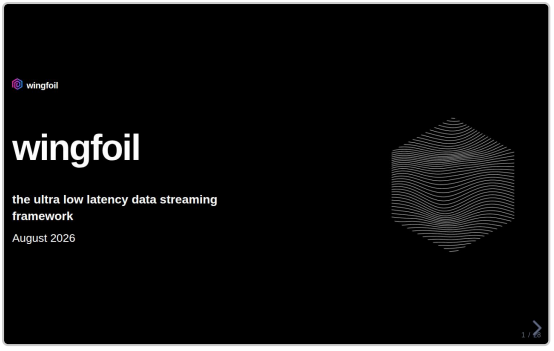
wingfoil — speaker notes

28 slides · printed fallback for the reveal.js speaker view (press S)



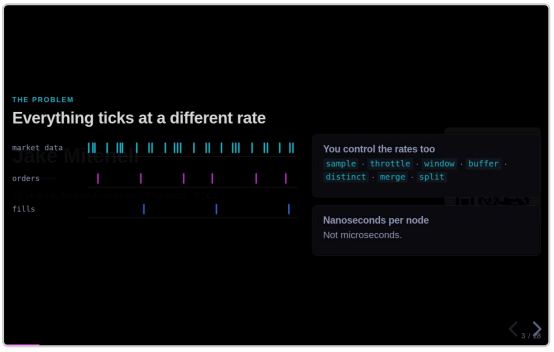
1 / 28

- Thanks for coming. Jake Mitchell. Open source project — **wingfoil**.
- Three parts:
- **1** — background: me, the kind of problem it's for, what it is, how you use it, quick tour.
- **2** — **Nitro**: the capability we wanted, why it meant rewriting the framework, the pattern we found, what it bought.
- **3** — what's next: down the stack, up the stack, where I need help.
- *[Move on. Don't linger.]*

2 / 28

Jake Mitchell

- 25 years in financial services.
- JPMorgan — electronic trading, options: rates, FX, precious metals.
- B2C2 now — systematic crypto options.
- Land this: **these problems aren't hypothetical**. 25 years of having them, where getting it wrong costs money.
- QR goes to the repo. Up again at the end.
- *[45 seconds. Don't dwell on your own CV.]*

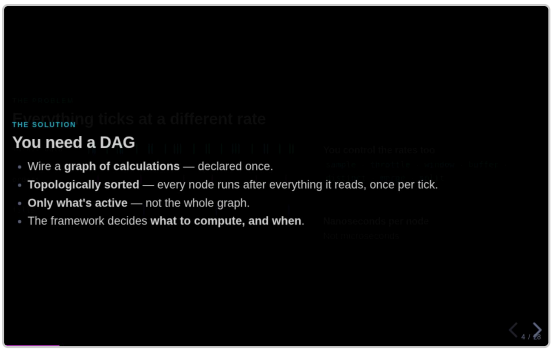


3 / 28

THE PROBLEM

Everything ticks at a different rate

- You're building something that reacts to information — and it **doesn't arrive politely**.
- Market data: when the venue sends it. Orders: when you act. Fills: when the venue decides.
- Three feeds, three rates, nothing lines up.
- And you have to control the rates yourself — sample, throttle, window, drop repeats.
- Budget is **nanoseconds per node**. Not micro. So the *framework's* overhead is what you pay for, not your arithmetic.
- *[If pushed on "burst": messages sharing a timestamp. One order sweeping three levels = three fills at one instant. That's the data, not you being slow.]*



4 / 28

THE SOLUTION

You need a DAG

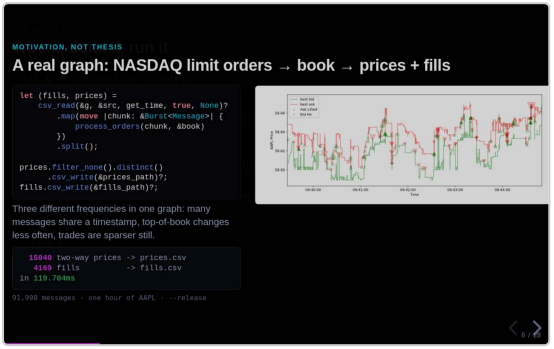
- What you want is a **DAG**.
- Declare the graph once. Not a callback at a time.
- Topologically sorted — every node runs after everything it reads, once per tick.
- Only what's **active**, not the whole graph.
- Framework works out what to compute and when. That's its job.

5 / 28

THE SURFACE

Wire it, build it, run it

- Smallest complete program.
- Builder → ticker → count → format → print. Build it, run it.
- **That output is real.** That's what it prints.
- Note the run mode — historical, so it finishes instantly. Five ticks at 100ms, no waiting.
- Same wiring runs live.

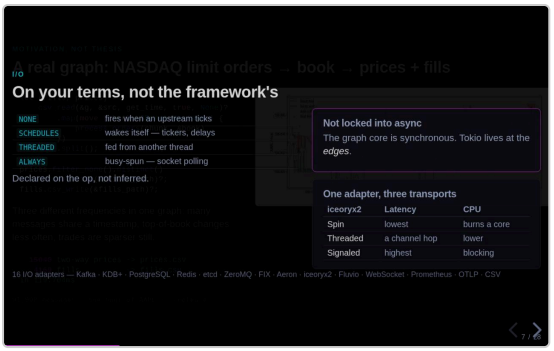


6 / 28

MOTIVATION, NOT THESIS

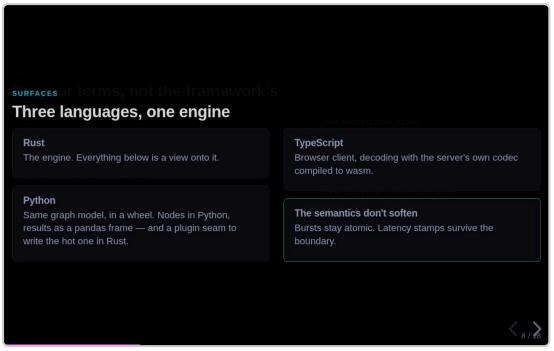
A real graph: NASDAQ limit orders → book → prices + fills

- Something real. CSV of NASDAQ limit orders → order book → trades and two-way prices → back out.
- Three frequencies in one graph: messages fast, top of book less often, trades sparser still.
- **An hour of AAPL. 92,000 messages. About a tenth of a second.**
- Same wiring points at a live feed.



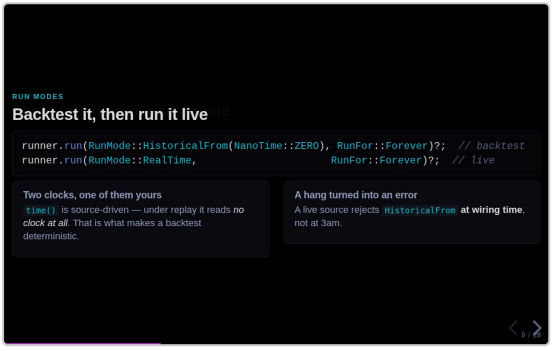
On your terms, not the framework's

- This is where most frameworks decide *for* you.
- A node **declares** how it's driven: on an upstream tick · wakes itself (tickers, delays) · fed from a thread · busy-spun (socket polling).
- Declared on the op. Nobody's inferring it from a name.
- Graph core is **synchronous**. Tokio at the edges. You don't pick one runtime for the whole process.
- Proof: same adapter, three transports. Spin = lowest latency, burns a core. Threaded = a channel hop. Signaled = blocks, costs nothing. **You pick.**



Three languages, one engine

- Rust is the engine.
- Python — same graph model in a wheel. Nodes in Python, pandas out, seam to write the hot one in Rust.
- TypeScript — browser end, decodes with the server's own codec compiled to wasm. No second implementation to drift.
- The bit that matters: **semantics don't soften at a language boundary.** Same bursts, same latency stamps.



Backtest it, then run it live

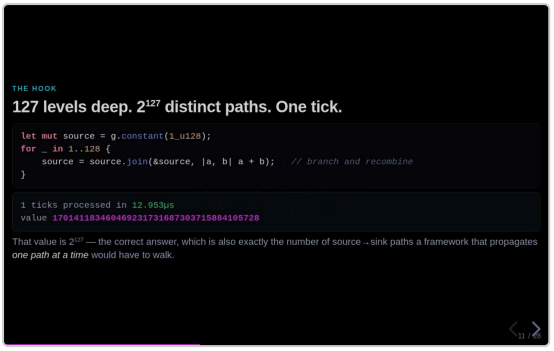
- The one most people actually want.
- Same wiring. Change the run mode.
- **Two clocks.** Engine time is source-driven — under replay it reads *no clock at all*. That's the determinism.
- Separate wall clock for latency stamping. Never branch logic on it.
- A live source **refuses a historical run at wiring time**. Historical collects input up front, so a live tail would deadlock.
- *[Difference between finding it in ten seconds and finding it at 3am.]*

10 / 28

PART TWO

Nitro

- That's part one.
- Part two — Nitro.

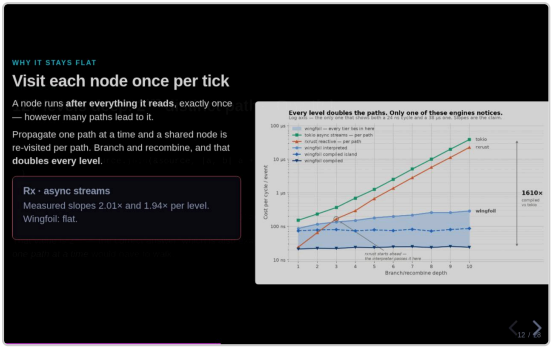


11 / 28

THE HOOK

127 levels deep. 2127 distinct paths. One tick.

- 128 levels deep. Branches and recombines at every level.
- So distinct source→sink paths **double every level**.
- At the bottom: 2^{127} paths.
- That number on screen is the answer — **and** the path count.
- **Thirteen microseconds. One tick.**
- *[Slow down. Let it sit.]*

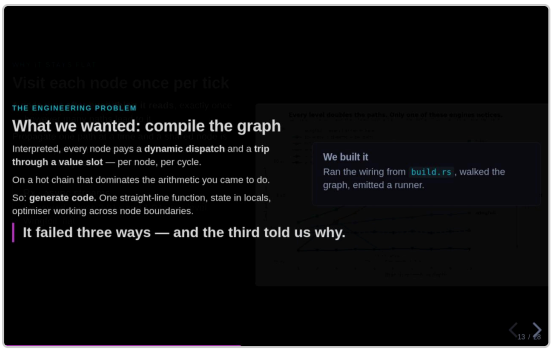


12 / 28

WHY IT STAYS FLAT

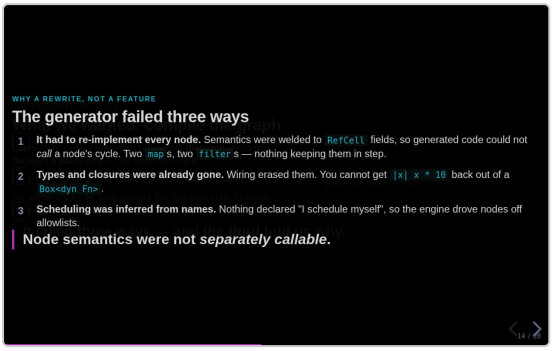
Visit each node once per tick

- The reason is boring, and that's the point.
- Every node runs once per tick, after everything it reads. However many paths lead in.
- Push one path at a time — reactive libraries, async streams — and you re-visit shared nodes per path. Cost doubles every level.
- Measured slopes: **2.01** and **1.94**. Ours: flat.
- *[Get to the caveats first: rxrust figure is HALVED from their raw number (their bench emits twice per sample). Tokio target is recursive Box::pin over a ready leaf — honest for per-path propagation, not the best async could do. **The claim is the slope, not the multiplier.**]*



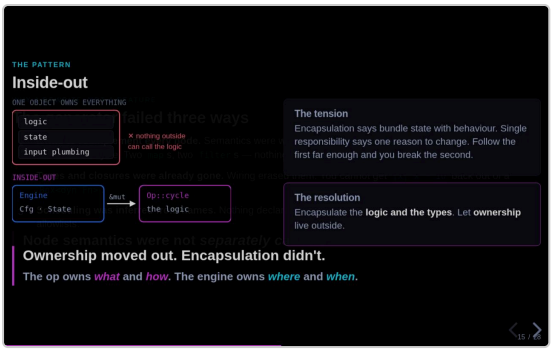
What we wanted: compile the graph

- Interpreted engine is fine, but there's overhead I don't love.
- Per node, per cycle: a **dynamic dispatch** and a **trip through a value slot**.
- On a hot chain that's bigger than the arithmetic you came to do.
- Obvious answer: **generate code.** One straight-line function, state in locals, optimiser working across node boundaries.
- Good plan. Every ingredient exists.
- So we built it. Wiring at build time, walk the graph, emit a runner.



The generator failed three ways

- **It failed. Three ways.**
- **1** — had to re-implement every node. Semantics welded to RefCell fields, so generated code couldn't *call* a node's cycle. Emitter carried its own copy, as strings. Two maps, two filters. Nothing keeping them in step.
- **2** — types and closures already gone. Types came back as name strings we parsed. You cannot get a closure back out of a Box dyn Fn. Workaround: user retypes every closure in a side table, with no compiler check the two agree.
- *[That one usually gets a laugh. Let it.]*
- **3** — nothing declared how it wanted scheduling. Engine drove nodes off **allowlists of names**.
- We deleted it. *[Pause.]*
- **The problem was never the generator. Node semantics weren't separately callable.** No cleverness in an emitter fixes a design with nothing to emit a call *to*.

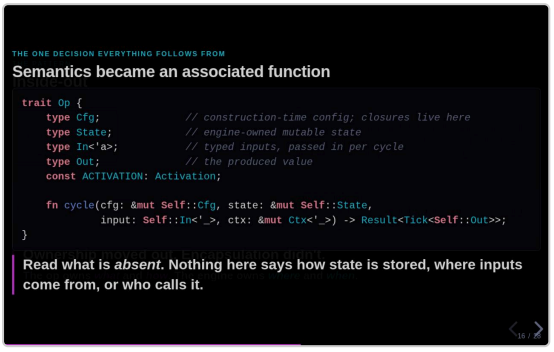


15 / 28

THE PATTERN

Inside-out

- The pattern that got us out.
- **The tension:** encapsulation says bundle state with behaviour. Single responsibility says one reason to change. Follow the first far enough and you break the second.
- Our node was computation *and* storage *and* plumbing. Three reasons to change, one object.
- **The resolution:** keep logic and types encapsulated. Move **ownership** out.
- *[Point at the arrow.]* The engine passes in state it *can't read*, to logic it *can't see inside*.
- **Ownership moved out. Encapsulation didn't.**
- **The op owns what and how. The engine owns where and when.**
- *["Isn't that just an ECS?" — yes, and reducers, gen_server, React hooks. Same family. Ours is the combination: reducer-shaped semantics + opaque associated types + a const for scheduling. That's what gets you two engines.]*

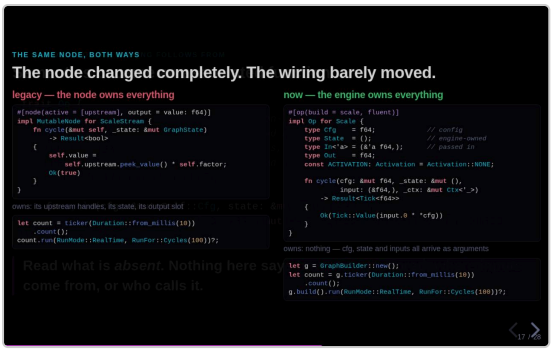


16 / 28

THE ONE DECISION EVERYTHING FOLLOWS FROM

Semantics became an associated function

- Four associated types and a function.
- Cfg (closures live here) · State (engine owns) · In (typed, per cycle) · Out. Plus a const for scheduling.
- **Notice what's missing.** Nothing says where state lives. Nothing says who calls it.
- It's a free function over explicit arguments.
- *[Return type, if asked: Tick has THREE states. Value ticks downstream, Quiet does nothing, Silent updates the value without ticking — what delay needs.]*

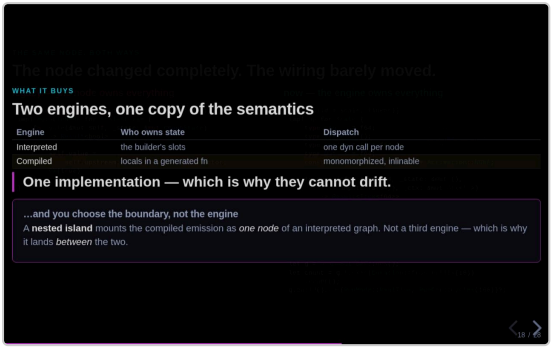


17 / 28

THE SAME NODE, BOTH WAYS

The node changed completely. The wiring barely moved.

- Same node, both ways.
- **Left:** &mut self, peeks a stored upstream handle, writes its own field. Owns its upstreams, its state, its output slot.
- **Right:** same node as an op. Owns **nothing** — everything arrives as an argument.
- Those two are unrecognisable.
- **Now the bottom row** — the wiring the user writes. Before and after.
- **One line different.** You hold a builder and make sources on it.
- We turned the engine inside out, and it cost anyone using it one line.

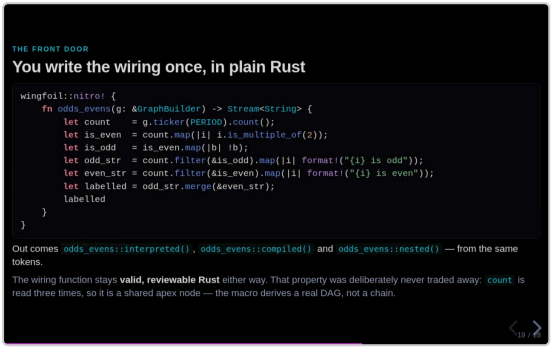


18 / 28

WHAT IT BUYS

Two engines, one copy of the semantics

- Two engines out of one definition.
- Interpreted: state in the builder's slots, one dyn call per node.
- Compiled: state in locals of a generated fn, monomorphized, optimiser inlines through.
- **One implementation of what a node does.** They can't drift — nothing to drift from.
- Island: mount the compiled version as one node *inside* an interpreted graph. Hot core compiled, edges open.
- **Not a third engine — the seam between the two.** Which is why it lands between them on every benchmark.

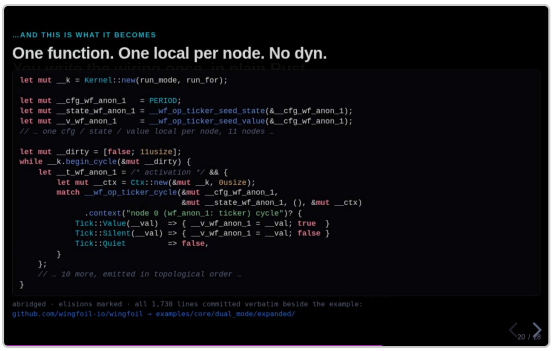


19 / 28

THE FRONT DOOR

You write the wiring once, in plain Rust

- You write the wiring once. Plain Rust.
- Real function. Valid, reviewable, readable in a PR.
- Out the other side: interpreted, compiled, nested. **Same tokens.**
- `count` used three times → a shared node. This is a real DAG, not a chain.
- *[Constraint, if asked: wiring must be straight-line. Values and closure logic can be as procedural as you like — the TOPOLOGY can't depend on a runtime value, because compiled monomorphizes one local per node.]*

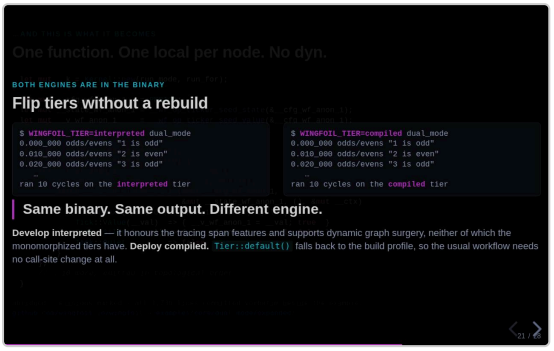


20 / 28

…AND THIS IS WHAT IT BECOMES

One function. One local per node. No dyn.

- What it turns into.
- Kernel on the stack. One local per node — cfg, state, value. Then a loop.
- *[Point at the call.]* **That's the same cycle function the interpreter calls.** Not a copy. A call.
- Wall number one, dead.
- Error context = the binding name *you* wrote, baked in as a static string. Costs nothing until something breaks.
- Whole expansion is committed. 1,730 lines. Don't take my word for it.

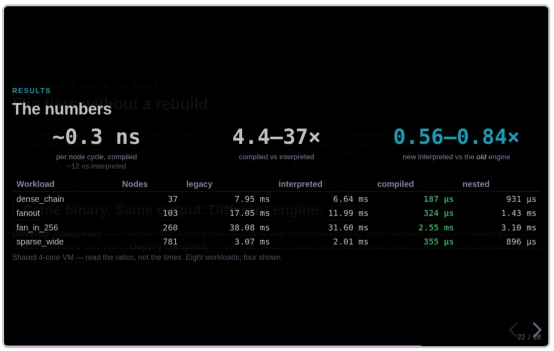


21 / 28

BOTH ENGINES ARE IN THE BINARY

Flip tiers without a rebuild

- Both engines are in the binary.
- Same binary, same output, different engine. **An environment variable.**
- Develop interpreted — proper tracing, surgery on a running graph. Neither of which compiled gives you.
- Deploy compiled.
- *[The two agreeing isn't a hope — a test runs the same graph through both and pins the outputs equal.]*

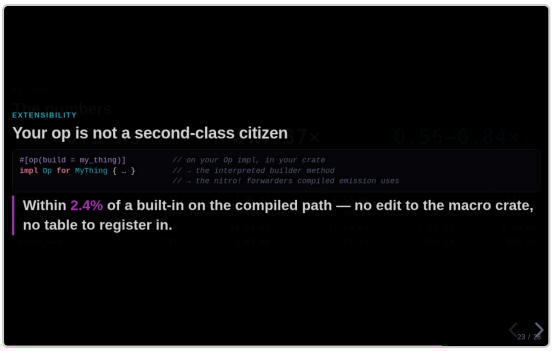


22 / 28

RESULTS

The numbers

- **~0.3 ns** per node cycle compiled. **~12 ns** interpreted.
- Compiled **4.4× to 37×**, depending on graph shape.
- The one I care about most: **new interpreted beats the OLD engine. All eight workloads.**
- Because most graphs keep running interpreted — and a rewrite that trades everyday performance for a fast path nobody uses is a bad trade.
- *[Read the ratios, not the times. Shared cloud VM, measured back to back in one run.]*

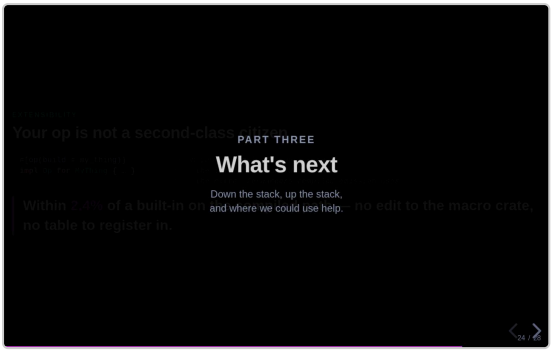


23 / 28

EXTENSIBILITY

Your op is not a second-class citizen

- The one I doubted most.
- Attribute on *your* op, in *your* crate → interpreted builder method + compiled forwarders.
- **Within 2.4% of a built-in** on the compiled path.
- No edit to the macro crate. No table to register in.
- There *is* no per-op table. Yours and ours take the same road.

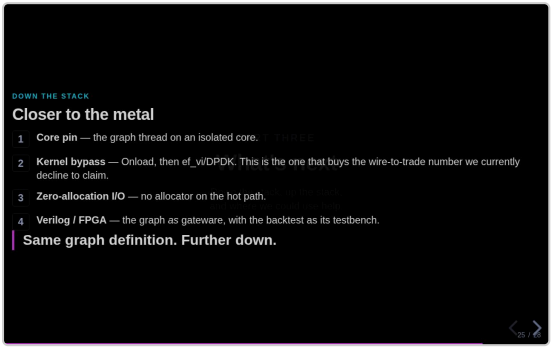


24 / 28

PART THREE

What's next

- That's Nitro.
- Last bit — where this goes.

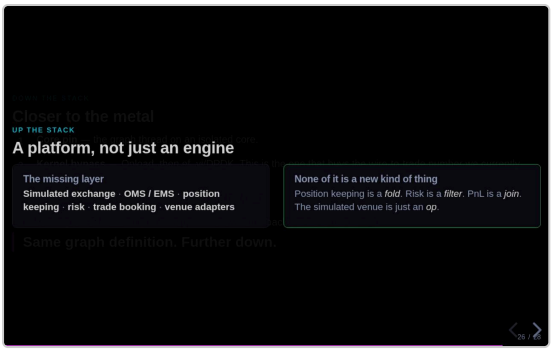


25 / 28

DOWN THE STACK

Closer to the metal

- **Core pin** — graph thread on an isolated core. Working implementation sits in an example; needs promoting into the runtime.
- **Kernel bypass** — Onload, then raw ef_vi/DPDK. Gets us a wire-to-trade number, which **I won't claim yet because I haven't measured it**.
- **Zero allocation** on the I/O path.
- **Verilog** — the graph as gateware, backtest as testbench. Exploratory. Don't oversell it.
- The point: **none of these change your wiring.** Same graph definition, further down. That's the payoff from part two.

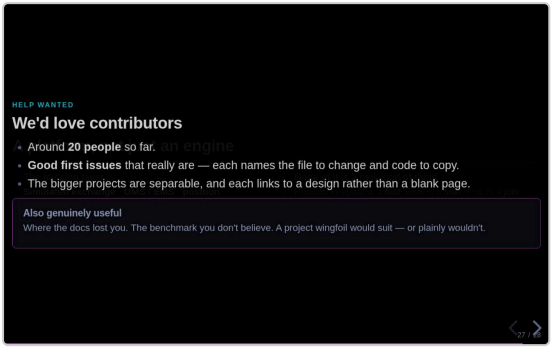


26 / 28

UP THE STACK

A platform, not just an engine

- Where most of the value is, for most people.
- **Be straight:** wingfoil is a compute engine. It is *not* a trading platform.
- Missing: simulated exchange, OMS, position keeping, risk, booking, more venue adapters.
- Simulator is the hard one — a naive fill-at-touch simulator lies to you, and you'll believe it.
- **The bet:** none of it is a new kind of thing. Position keeping is a *fold*. Risk is a *filter*. PnL is a *join*. The sim venue is just an *op* — RunMode picks.
- Ordinary graph vocabulary. Not a second framework bolted on top.

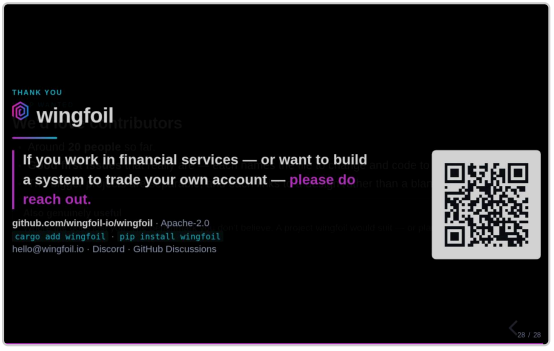


27 / 28

HELP WANTED

We'd love contributors

- The ask.
- ~20 contributors so far. I'd love more.
- **The good first issues are actually good first issues** — each names the file to change and code to copy. I know that label's been ruined elsewhere.
- Bigger projects are separable — one person each, and each links to a design not a blank page.
- Most useful isn't always code: where the docs lost you · which benchmark you don't believe · a project this would suit, or obviously wouldn't.



28 / 28

THANK YOU

wingfoil

- **If you work in financial services — or you're building something to trade your own account — please get in touch.** That's genuinely why I'm up here.
- Repo's on the QR. Thank you.
- *[Leave it up. Don't black the screen — let people scan.]*
- *[If it's quiet: the question I usually get is what happens when the graph shape isn't known until runtime. That's Project Lightning, and it's open.]*
